华东师范大学软件学院实验报告

Lab3 IPv4 数据报与网络路径

实验课程：计算机网络	年级：2024 级	实验成绩：
实验名称: IPv4	姓名：刘小凡	实验日期：2026-10-13
实验编号：Lab3	学号：10244602411	实验时间：2 学时

一、实验目的

本实验通过真实 HTTP 流量和逐 TTL 的 ICMP 探测，分析 IPv4 首部结构、分片判据与网络路径，并独立验算接收报文的首部校验和。具体目标如下。

1. 掌握在 Wireshark 中选择联网接口、设置 tcp port 80 捕获过滤器、开启网络名称解析并关闭混杂模式的完整抓包流程，能够保存可复核的 pcapng 迹线；

2. 能够把 IPv4 首部各字段与被高亮的原始字节逐一对应，理解哪些字段共享同一个字节，并据此还原首部长度字段 IHL 的编码方式；

3. 通过实测数据总结 Total Length、Identification、TTL 以及分片标志的变化规律，并区分“未分片”的判据与 IP 校验和卸载等容易误判的现象；

4. 使用 traceroute 类探测发现从本机到远端服务器的路由器级路径，理解星号跳的含义，并能借助地址登记信息为每一跳标注归属组织；

5. 独立完成一次 IPv4 首部校验和的 1 的补码求和验算，把 10 个 16 位字与对应字段一一列出，验证结果等于 0xffff。

二、实验内容与实验步骤

实验按手册的五个步骤进行：捕获 HTTP 迹线；展开 IPv4 首部并核对字节；绘制字节布局并回答六道问题；根据同一目标服务器的逐跳探测绘制路径；选取接收方向、首部 20 B 且校验和非零的报文完成验算。

提交内容分别为 IPv4 结构图和六题答案、网络路径图及实际探测输出、校验和的十个 16 位字与求和过程，均在第四节及附录中给出。

三、实验环境

表 1　实验环境与通信端点

项目	实测配置
操作系统	Windows 11 25H2（build 26200），64 位
抓包与分析	Wireshark / TShark 4.6.8，Npcap 1.88
命令行工具	curl 8.21.0；Windows tracert；系统 ICMP API
捕获接口	WLAN：MediaTek Wi-Fi 6E MT7922 (RZ616) 160MHz Wireless LAN Card
接口地址	IPv4 172.31.165.86；网关 172.31.0.1；本机 MAC 14:ac:60:56:df:85
链路下一跳 MAC	54:c6:ff:7b:38:02
目标服务器	www.baidu.com → 182.61.200.108（用 --resolve 固定）
链路层类型	Ethernet（关闭混杂模式，启用网络名称解析）

四、实验过程与分析

4.1 HTTP 流量的获取与定位

本次使用 curl 获取网页。参数 -4 强制使用 IPv4，--resolve 固定 www.baidu.com 的服务器地址，--interface 绑定 WLAN 源地址，--noproxy 禁用 curl 的显式代理；这些参数并不关闭系统 TUN，路径探测的影响另见 4.5 节。命令按 Windows PowerShell 的续行语法书写如下。

```
curl.exe -4 --noproxy "*" --interface 172.31.165.86 `
  --resolve www.baidu.com:80:182.61.200.108 `
  -o baidu.html -w "%{http_code} %{size_download} %{time_total}\n" `
  http://www.baidu.com/
```

curl 返回 HTTP 200，实际下载 2381 B，用时约 0.855 s；保存的响应首部包含 Content-Length: 2381，与下载文件大小一致。随后在 Wireshark 选择 WLAN、Ethernet 链路层和 tcp port 80 捕获过滤器，关闭混杂模式并启用网络名称解析。tcp port 80 本身不限定 IP 版本，本次由 curl -4 产生 IPv4 流量。

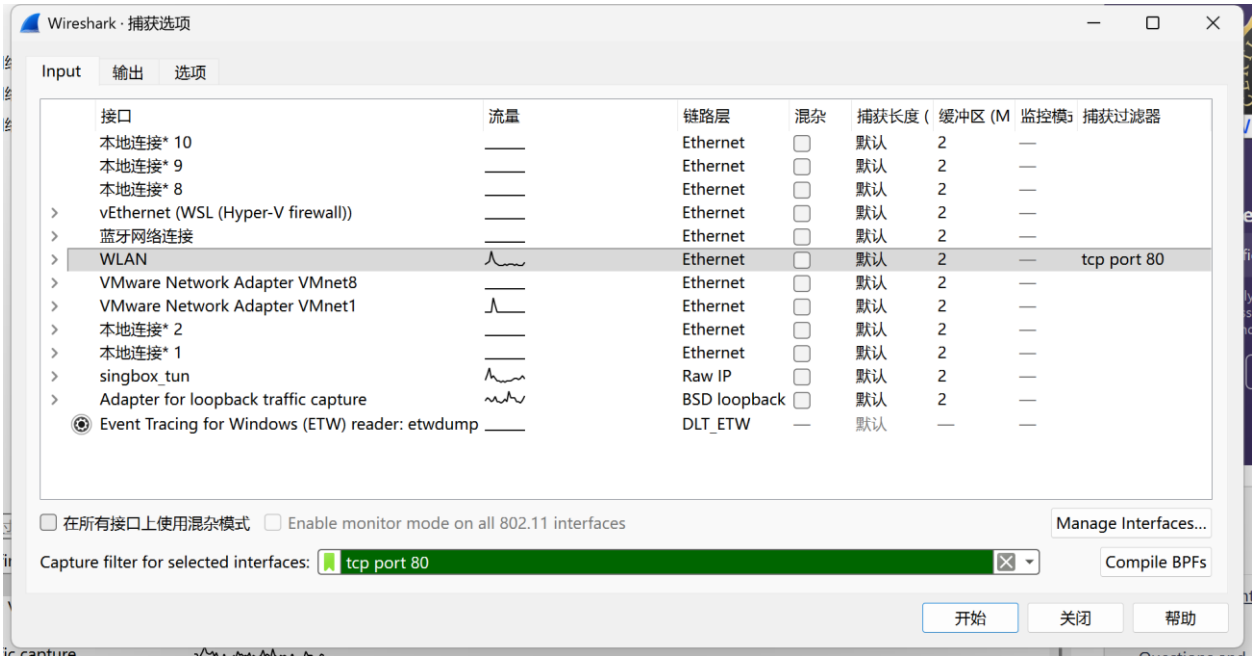


图 1　抓包前的实际设置：WLAN、Ethernet、tcp port 80，混杂模式关闭。

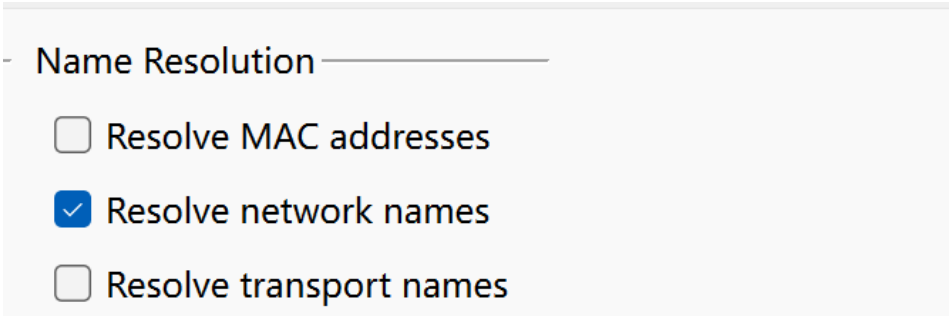


图 2　同一次捕获设置中 Resolve network names 已勾选；此图为原始窗口的局部截图。

开始捕获后执行上述 GET，结束后停止并保存迹线。原始 Lab3_HTTP.pcapng 共 108 帧；按 tcp.stream == 0 导出目标连接，得到 Lab3_HTTP_target.pcapng 的 19 帧，其中发送 10 帧、接收 9 帧。原始前 19 帧就是该连接，因此导出后的帧号保持一致。交互阶段见表 2。

表 2　目标连接的报文阶段

阶段	帧号	方向	IP Total Length	说明
握手	1–3	本机 ⇄ 服务器	52 / 52 / 40	SYN、SYN/ACK、ACK
请求	4	本机 → 服务器	117	HTTP GET（含 77 B 请求）
响应	5–9, 14	服务器 → 本机	40 / 1400 / 216 / 373 / 373 / 829	ACK 与数据段；第 9、14 帧被标记为重传
确认与挥手	10–13, 15–19	双向	52 / 40 / 60	Dup ACK、FIN/ACK、ACK

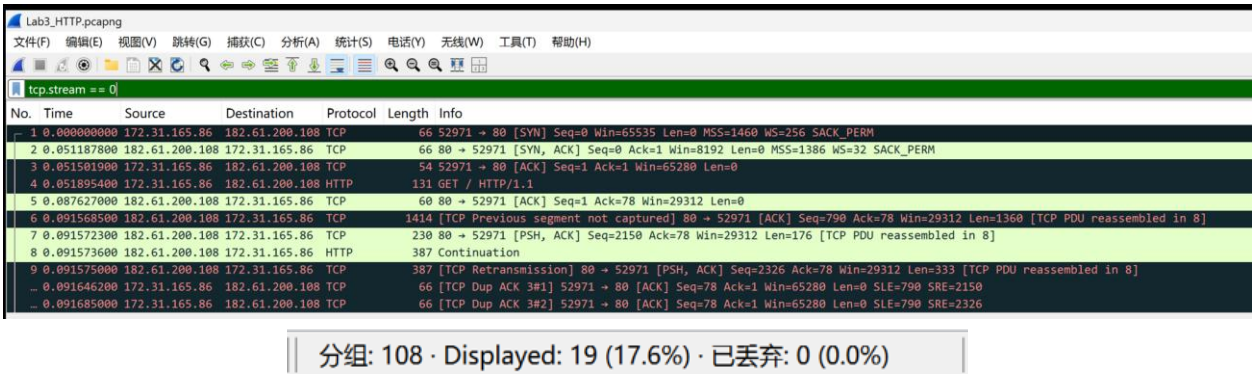


图 3　原始迹线中 tcp.stream == 0 的列表局部及同一视图统计栏：108 帧中显示 19 帧。



图 4　同一 TCP 流的实际内容：GET / HTTP/1.1、HTTP/1.1 200 OK 和 Content-Length: 2381。

列表中的 Previous segment not captured、Dup ACK 和 Retransmission 提示反映 TCP 序号、丢失或重传分析，不能等同于 IPv4 分片。此次传输最终成功，HTTP 响应的完整正文大小以 curl 保存的文件和响应首部核对。

4.2 IPv4 首部的逐字段检查

选取第 4 帧 HTTP GET，IPv4 首部长度为 20 B，不含选项。图 5 显示实测字段及原始字节，表 3 将字段与首部内偏移逐项对应。偏移从 IPv4 首部第一个字节起算。

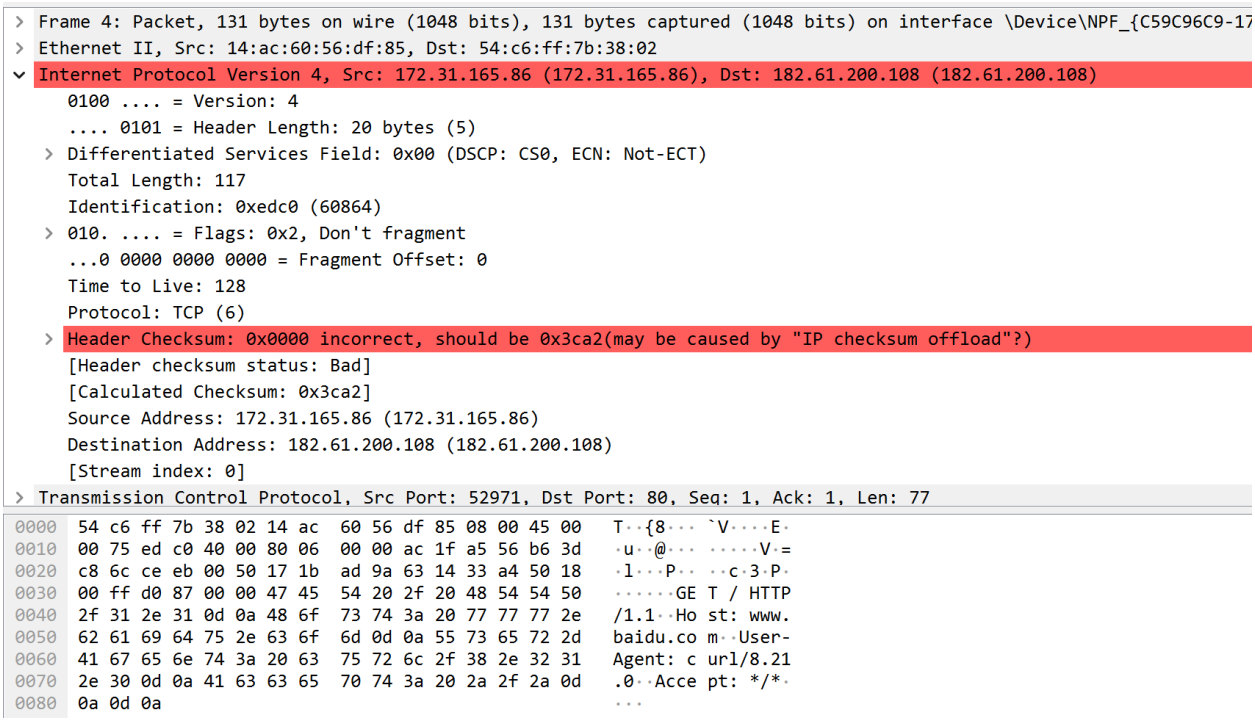


图 5　第 4 帧 HTTP GET：Total Length=117 B、TTL=128，捕获到的首部校验和为 0x0000。

表 3　第 4 帧 IPv4 首部字段与字节偏移

字段	字节	长度	实测值	说明
Version + IHL	0	1 B	0x45	Version=4，IHL=5
Differentiated Services	1	1 B	0x00	DSCP CS0，ECN Not-ECT
Total Length	2–3	2 B	0x0075 = 117	首部 20 + 载荷 97
Identification	4–5	2 B	0xedc0	分片重组时用于归组
Flags + Fragment Offset	6–7	2 B	0x4000	DF=1，MF=0，Offset=0
Time To Live	8	1 B	128	本次 HTTP 发送方向的初值
Protocol	9	1 B	6	上层为 TCP
Header Checksum	10–11	2 B	0x0000	捕获值为零，与发送端校验和卸载现象相符
Source Address	12–15	4 B	172.31.165.86	本机 WLAN
Destination Address	16–19	4 B	182.61.200.108	远端服务器

本次 HTTP 发送方向的首部校验和在抓包处均为 0x0000，Wireshark 提示可能由 IP checksum offload 引起。这与发送端抓包发生在网卡填写校验和之前的现象相符[6]，不能据此认定线上报文错误，也不能仅凭零值证明每一帧的硬件处理过程。接收方向校验和非零且验证通过，4.6 节选取第 5 帧做独立验算。

4.3 IPv4 数据报的字节布局

手册要求标出首部各字段的字节位置与长度，并合并共享字节的字段。图 6 将 20 B 首部按每行 4 B 展开；下方完整帧条带按字节数等比绘制，区分以太网首部、IPv4 首部、TCP 首部和 HTTP 请求。

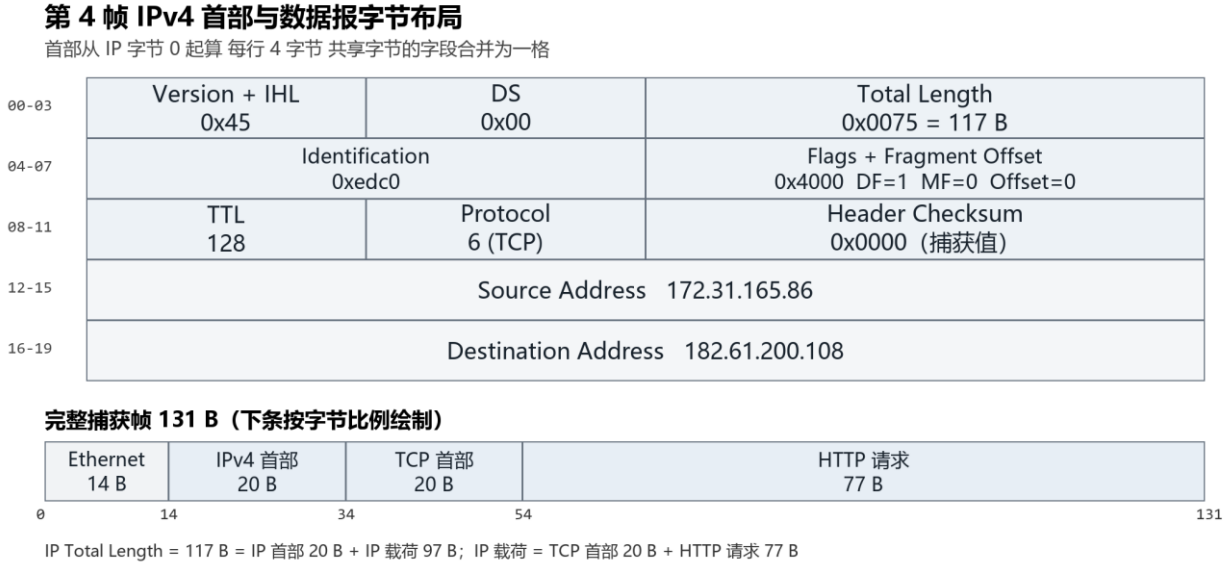
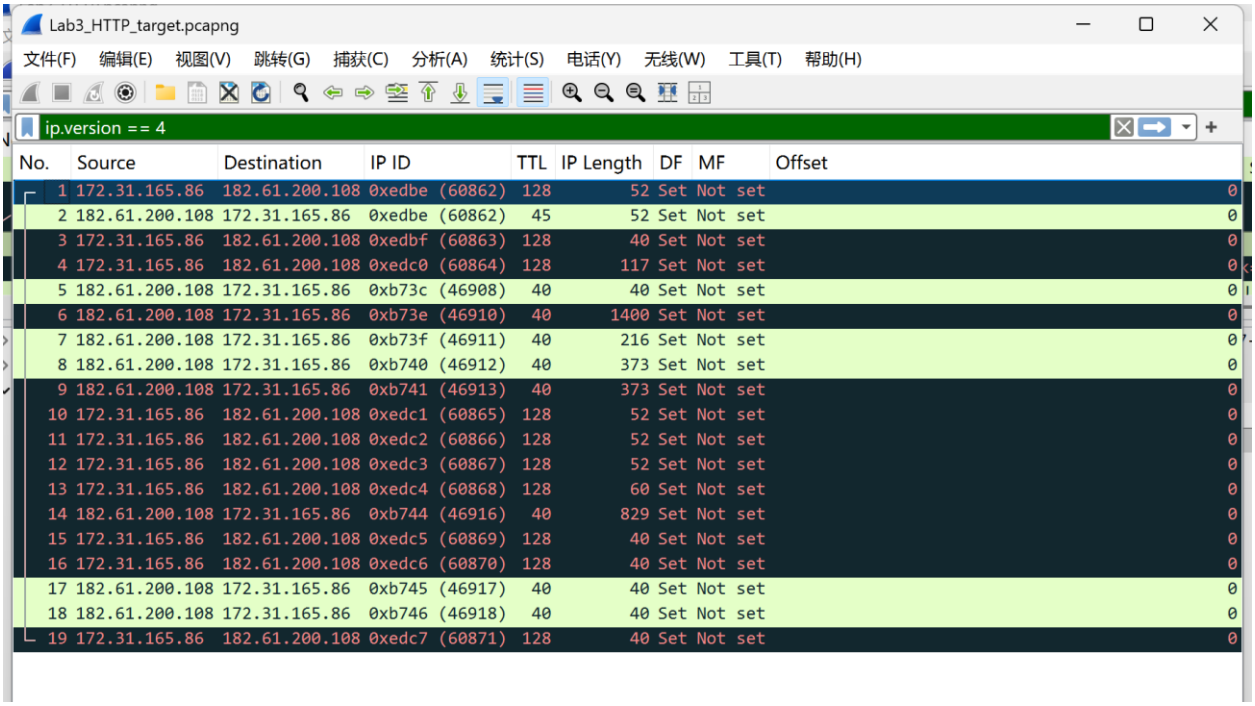


图 6　依据第 4 帧绘制的字节布局：帧长 131 B，IP 总长度 117 B，IP 载荷 97 B；本图为示意图。

Version 与 IHL 是两个独立的 4 位字段，共享第 0 字节；Flags 的 3 位与 Fragment Offset 的 13 位共享第 6、7 字节。不能把“共享字节”写成“不是独立字段”。本次无 Options，IPv4 首部占帧内偏移 14–33，TCP 从帧内偏移 34 开始。

4.4 六道问题的回答

图 7 把同一目标连接的 Identification、TTL、总长度、DF、MF 与 Offset 列在同一视图中，以下据此回答手册的六道问题。



No.	Source	Destination	IP ID	TTL	IP Length	DF	MF	Offset
1	172.31.165.86	182.61.200.108	0xedbe (60862)	128	52	Set	Not set	0
2	182.61.200.108	172.31.165.86	0xedbe (60862)	45	52	Set	Not set	0
3	172.31.165.86	182.61.200.108	0xedbf (60863)	128	40	Set	Not set	0
4	172.31.165.86	182.61.200.108	0xedc0 (60864)	128	117	Set	Not set	0
5	182.61.200.108	172.31.165.86	0xb73c (46908)	40	40	Set	Not set	0
6	182.61.200.108	172.31.165.86	0xb73e (46910)	40	1400	Set	Not set	0
7	182.61.200.108	172.31.165.86	0xb73f (46911)	40	216	Set	Not set	0
8	182.61.200.108	172.31.165.86	0xb740 (46912)	40	373	Set	Not set	0
9	182.61.200.108	172.31.165.86	0xb741 (46913)	40	373	Set	Not set	0
10	172.31.165.86	182.61.200.108	0xedc1 (60865)	128	52	Set	Not set	0
11	172.31.165.86	182.61.200.108	0xedc2 (60866)	128	52	Set	Not set	0
12	172.31.165.86	182.61.200.108	0xedc3 (60867)	128	52	Set	Not set	0
13	172.31.165.86	182.61.200.108	0xedc4 (60868)	128	60	Set	Not set	0
14	182.61.200.108	172.31.165.86	0xb744 (46916)	40	829	Set	Not set	0
15	172.31.165.86	182.61.200.108	0xedc5 (60869)	128	40	Set	Not set	0
16	172.31.165.86	182.61.200.108	0xedc6 (60870)	128	40	Set	Not set	0
17	182.61.200.108	172.31.165.86	0xb745 (46917)	40	40	Set	Not set	0
18	182.61.200.108	172.31.165.86	0xb746 (46918)	40	40	Set	Not set	0
19	172.31.165.86	182.61.200.108	0xedc7 (60871)	128	40	Set	Not set	0

图 7　目标连接全部 19 帧的双向字段：MF 均未设置、Offset 均为 0；IP ID 与 TTL 的变化可直接比较。

1 本机与远端 IPv4 地址

本机 WLAN 地址为 172.31.165.86，目标服务器为 182.61.200.108；收发方向由源、目的地址的顺序区分。这是本机抓包点看到的地址，不由此断言服务器看到的本机地址，NAT 可能改写地址。

2 Total Length 的统计范围

Total Length 包含 IPv4 首部及其载荷，不含以太网首部。第 4 帧：117 B = 20 B IP 首部 + 20 B TCP 首部 + 77 B HTTP 请求；帧长 131 B 与 IP 总长度之差为 14 B，恰好是本次 Ethernet II 首部。

3 Identification 字段的变化

本机方向依次为 0xedbe 至 0xedc7，逐包加 1；接收方向为 0xedbe、0xb73c、0xb73e、0xb73f、0xb740、0xb741、0xb744、0xb745、0xb746，存在跳号。两方向在第 1、2 帧还出现了相同值。因此本次 ID 不在连接内恒定，也不保证双向不同；不能把观察到的递增趋势推广为协议要求。ID 用于分片重组，不是 TCP 连接号。RFC 6864 所称 atomic datagram 满足 DF=1、MF=0、Offset=0，允许其 ID 重复[2]；本次 19 帧满足这一条件。

4 发送方向 TTL 的初值

本次 HTTP 连接的 10 个发送包在 WLAN 抓包处 TTL 均为 128，低于 8 位字段的最大值 255。路径探测则故意使用 TTL 1–17。接收 SYN/ACK 的 TTL 为 45，其余接收包为 40；未确认对端初始 TTL，不能用 64−接收 TTL 直接推断路由跳数。

5 未分片报文的判据

MF=0 且 Fragment Offset=0 表示本包未分片。19 帧均满足该判据。DF=1 是禁止后续分片的指示，不是判断未分片的必要条件；DF=0 的短包也可能不分片。MF=0 但 Offset>0 是最后一片。Fragment Offset 以 8 B 为单位；图 7 的 Offset=0 在两种单位下数值相同。

6 IHL 的长度编码

本次目标连接首部均为 20 B。首字节 0x45 的高 4 位 0100 表示 Version=4，低 4 位 0101 表示 IHL=5。IHL 以 32 位字即 4 B 为单位，5×4=20 B；合法的最小值为 5，4 位最大值 15 对应 60 B，选项与填充最多共 40 B。

4.5 Internet 路径与隧道影响的排查

首先对与 HTTP 相同的服务器运行 tracert -4 -h 30 -w 700 182.61.200.108，原始输出只显示两跳：

```
1    *        62 ms    *        182.61.200.108
2    33 ms    32 ms    33 ms    182.61.200.108
```

两跳结果本身不能证明异常，因为目标也可能离源很近；关键证据是对应 WLAN 迹线中 Echo Request 的 TTL 全为 128，未体现 tracert 的逐 TTL 探测。结合 singbox_tun 的默认路由指向 172.18.0.2，可判断该结果受到本机隧道影响，不能据此描述 WLAN 的逐跳转发路径。排查未修改 VPN 或系统路由。

随后通过 Windows IcmpSendEcho2Ex API 绑定 SourceAddress=172.31.165.86，每跳发送 3 次探测，单次超时 900 ms。保存 API 返回结果，并在 WLAN 抓包核对实际 TTL。下文称其为“绑定 WLAN 源地址的 traceroute”，不把脚本输出冒充为 tracert.exe 的输出[4]。

逐 TTL 探测使不同距离的路由器返回 ICMP Time Exceeded。第 5 帧的外层源地址为 10.100.4.1、目的为本机，ICMP Type=11、Code=0；内部引用原始请求的 IPv4 首部与 ICMP 首部，供发送方匹配探测，见图 8[3]。

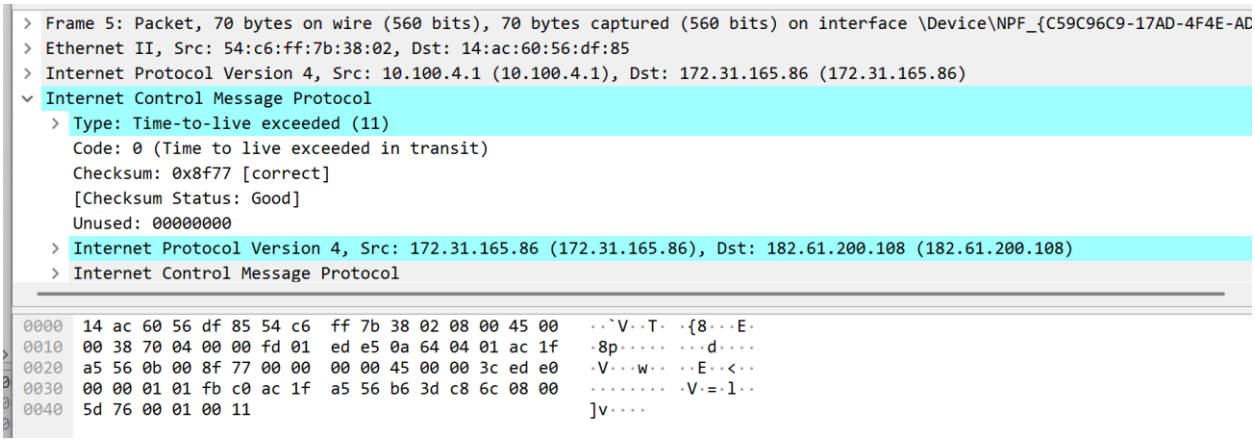


图 8　路径迹线第 5 帧：路由器 10.100.4.1 返回 Type 11 / Code 0，并引用本机发往服务器的原始请求。

实际探测在第 17 跳收到目标回复。76 帧中有 51 个 Echo Request、23 个 Time Exceeded 和 2 个 Echo Reply。TTL 1–17 各对应 3 个发出的探测。仅使用 icmp.type == 8 会同时匹配超时报文

引用的内层 Echo 请求；图 9 的过滤条件同时排除了外层 Type 11，因此显示 51 帧，而不是包含引用的 74 帧。

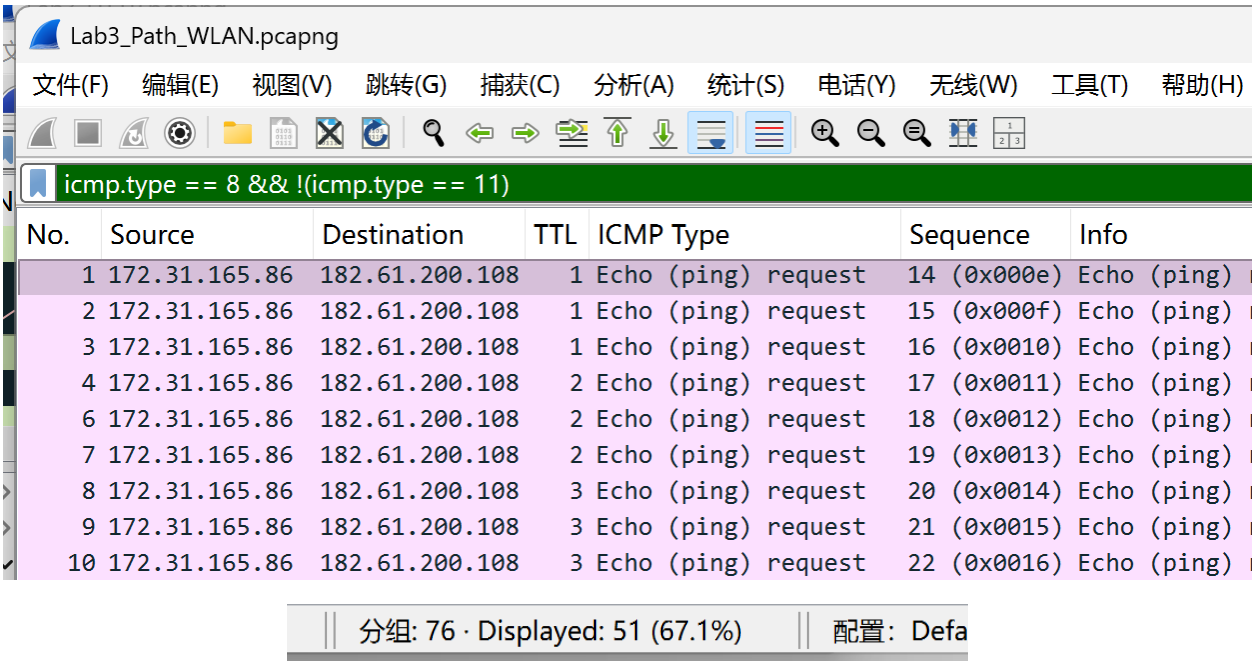


图 9　TTL 1、2、3 各三次 Echo Request 的列表局部，以及同一视图的统计栏；过滤后显示 51 个请求。

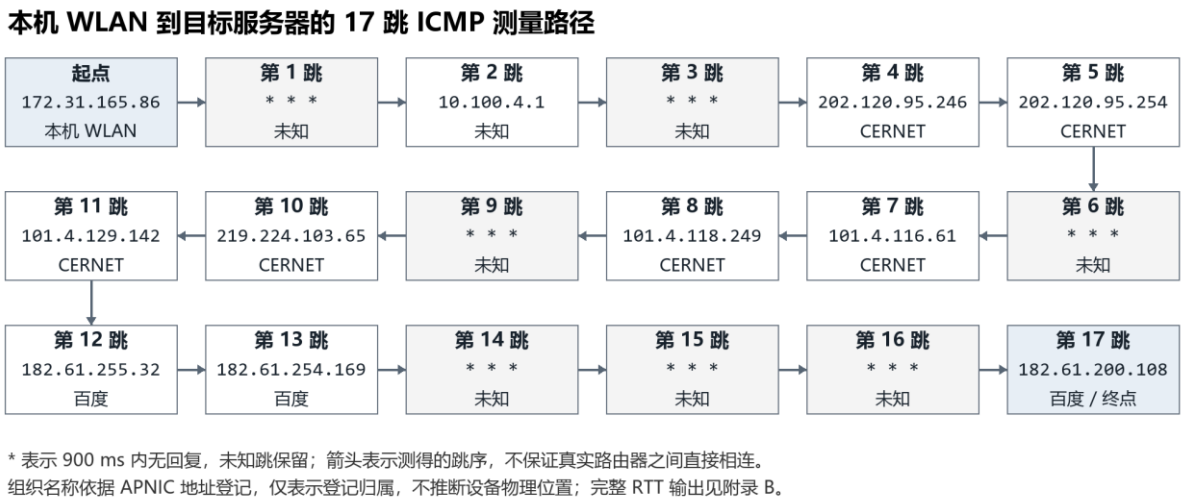


图 10　依据 source_trace.json 绘制的 17 跳测量路径，含本机、目标服务器与未知跳；本图为示意图。

本次 PTR 反向解析未成功，组织标注使用 APNIC RDAP 的地址登记资料，不能当作设备位置证据。202.120.95.246 所在登记范围为 202.112.0.0–202.121.255.255，不能误写成单一的 202.112.0.0/13；101.4.116.61 属于 101.4.0.0/14，219.224.103.65 属于 219.224.0.0/15，182.61.255.32 属于 182.61.0.0/16。私网地址 10.100.4.1 的组织保持未知。查询结果与完整 RTT 输出分别见附录 A、B。

路径图中的星号只表示在规定的 900 ms 内没有收到对应回复。它不能证明该跳不存在，也不能仅凭它断言那台路由器关闭了 ICMP；常见原因还包括路由器限速 ICMP、回复走了不同路径或优先级较低而被丢弃。另外，ICMP 探测出来的路径不保证与 TCP 数据实际走的路径完全相同，反向路径也可能不同，所以这张图描述的是本次测量条件下的转发路径。

可确认的是：所响应的公网地址先后落在 CERNET 与百度的登记范围，目标 182.61.200.108 在第 17 跳回复。现有 PTR 和登记资料不能确认具体校园设备、上海交通大学节点或路由器地理位置，因此不作这类推断。

4.6 接收包 IPv4 首部校验和验算

按手册要求选择目标连接第 5 帧：接收方向、首部 20 B、校验和非零。其源为 182.61.200.108、目的为 172.31.165.86，实收首部校验和 0xcb73，Wireshark 标记 [correct]。图 11 高亮了这 20 B；首部字节如下。

```
45 00 00 28 b7 3c 40 00 28 06 cb 73 b6 3d c8 6c ac 1f a5 56
```

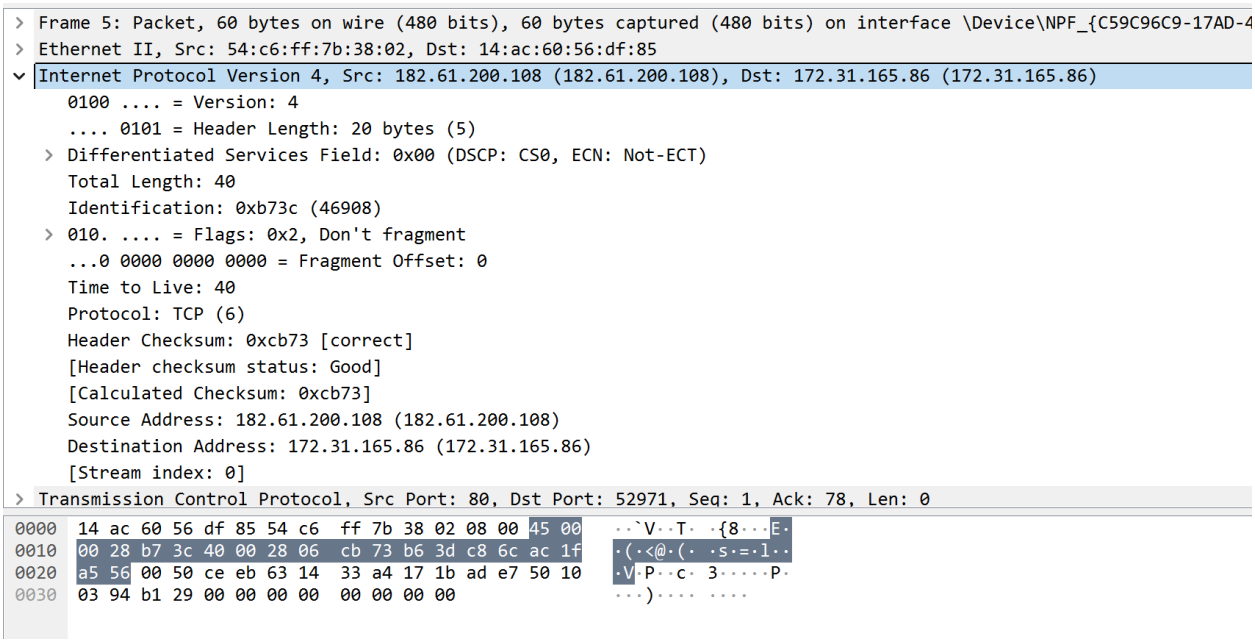


图 11　目标连接第 5 帧的 20 B IPv4 首部高亮，实收校验和 0xcb73，经 Wireshark 验证为 correct。

按网络字节序将首部分成十个 16 位字，包含实收校验和字段进行普通加法，再将进位回卷，见表 4。

表 4　接收首部的十个 16 位字与普通累计和

序号	16 位字	对应 IPv4 字段	累计和
1	0x4500	Version、IHL、DS	0x4500
2	0x0028	Total Length	0x4528
3	0xb73c	Identification	0xfc64
4	0x4000	Flags、Fragment Offset	0x13c64
5	0x2806	TTL、Protocol	0x1646a
6	0xcb73	首部校验和（实收值）	0x22fdd
7	0xb63d	源地址前半：182.61	0x2e61a
8	0xc86c	源地址后半：200.108	0x3ae86
9	0xac1f	目的地址前半：172.31	0x45aa5
10	0xa556	目的地址后半：165.86	0x4fffb

普通和为 0x4fffb。高 16 位 0x0004 加回低 16 位 0xfffb，得到 0xffff；再取反为 0x0000，完成接收验证。此求和只覆盖 IPv4 首部，不覆盖 TCP 或 HTTP 载荷[1]。

接收验证：`0x4fffb → 0xfffb + 0x0004 = 0xffff → 取反 = 0x0000`。

另将首部校验和字段临时置零进行生成复算：普通和 0x43488，回卷得到 0x348c，取反为 0xcb73。该值与原始报文及 Wireshark 结果一致。置零仅用于计算，未改写抓包文件。

生成复算：`0x43488 → 0x3488 + 0x0004 = 0x348c → 取反 = 0xcb73`。

上述字节和求和结果已通过 analyze.py 从 pcapng 重新读取并独立核验，与 GUI 显示相互印证。第 5 帧 Ethernet 帧长为 60 B，而 IP 总长度为 40 B；14 B 链路首部之外的 6 B 填充不参与 IP 首部校验和计算。

五、实验结果总结

5.1 实验结论

关于 IPv4 首部：本次所有报文首部长度均为 20 B，无选项字段；首字节 0x45 同时编码版本号与首部长度，IHL 以 4 字节为单位，最大值 15 对应 60 B。Total Length 统计 IP 首部与载荷之和，不含链路层首部。

关于标识与分片：本机方向 Identification 逐包递增，接收方向存在跳号，说明它只是分片重组时用于归组同一数据报的标识，并非连接号；19 个包全部满足 MF=0 且 Offset=0，因此都未分片，其中 DF=1 是发送方的策略选择而不是未分片的判据。

关于 TTL：本机普通数据流的 TTL 初值为 128，低于 8 位字段上限 255；接收方向报文 TTL 为 40，首个响应为 45。这里只报告观测值——由 64 减 TTL 直接推算跳数是不严谨的，因为无法确认对端的初始 TTL 一定是 64，而且双向路由可能不同。

关于校验和：接收包首部以实收值求和回卷得到 0xffff，置零生成复算得到 0xcb73；发送包捕获值为零与校验和卸载现象相符，不能据此判定线上错误。

关于与教材的差异：结构上没有发现差异，但有一处实现层面的现象值得记住——协议规定的字段未必在抓包点就已填好，校验和卸载就是典型例子。这提醒我们在用抓包结论判断“对错”之前，要先确认抓包位置相对于协议栈处理的位置。

5.2 实验体会

本次实验中，发送包的零校验和和 tracert 的两跳结果都需要结合抓包位置分析。前者应与校验和卸载区分，后者需检查实际发出的 TTL。把工具提示与独立字段验证相结合，才能避免把本机处理现象误写为网络结论。

另一处体会是描述数据时要留有余地。Identification 在本机方向确实逐包递增，但接收方向有跳号，因此报告里只写“呈递增趋势”而不断言每包加一；星号跳也只写“规定时间内无应答”，不推断该路由器关闭了 ICMP。把结论限制在证据支持的范围内，比给出一个漂亮但过度的断言更可靠。

最后是独立复核的价值。校验和既可以让 Wireshark 判定，也可以自己按 1 的补码算一遍；两条路径得到同一个结果，才算真正验证。本报告的字段与统计量均由脚本从 pcapng 原始字节重新解析，与界面显示互为交叉验证，而不是从截图抄写。

六、附录

附录 A 迹线与验证文件

表 5　可复核的实验数据文件

文件	说明
Lab3_HTTP_target.pcapng	目标 HTTP 连接，19 帧，报告主要依据
Lab3_HTTP.pcapng	抓包原始迹线 108 帧（含背景流量），仅作留存
Lab3_Path_WLAN.pcapng	绑定 WLAN 源地址的路径探测，76 帧
Lab3_Traceroute.pcapng	普通 tracert 的 WLAN 迹线，隧道干扰证据
logs/Lab3_HTTP_target.tsv	逐帧字段提取结果
logs/Lab3_Path_WLAN.tsv	路径探测字段提取（-E occurrence=f）
logs/source_trace.json	17 跳路径结构，图 10 的数据来源
logs/rdap_*.json	APNIC 地址登记查询原始结果
facts.json / analyze.py	字段提取与校验和独立验算脚本及结果

附录 B 完整逐跳测量输出

以下为 source_trace.txt 保存的真实输出。每行对应一个 TTL，星号为超时；终点在第 17 跳收到回复。

```
IPv4 source-bound traceroute  172.31.165.86 -> 182.61.200.108
Windows IcmpSendEcho2Ex  |  3 probes per hop  |  timeout 900 ms
 1  *  *  *                  *
 2  21 ms  *  *              10.100.4.1
 3  *  *  *                  *
 4  5 ms  4 ms  6 ms         202.120.95.246
 5  7 ms  6 ms  6 ms         202.120.95.254
 6  *  *  *                  *
 7  66 ms  4 ms  6 ms        101.4.116.61
 8  6 ms  6 ms  5 ms         101.4.118.249
 9  *  *  *                  *
10  *  69 ms  68 ms          219.224.103.65
11  32 ms  30 ms  30 ms       101.4.129.142
12  *  *  33 ms              182.61.255.32
13  39 ms  30 ms  32 ms       182.61.254.169
14  *  *  *                  *
15  *  *  *                  *
16  *  *  *                  *
17  *  32 ms  30 ms          182.61.200.108
```

附录 C 复核命令

以下命令以 PowerShell 为例，反引号表示续行。抓包文件名按附录 A；绑定源地址的路径探测实现为 source_trace.py。

```
tshark.exe -r Lab3_HTTP_target.pcapng -Y "frame.number == 5" `
  -o ip.check_checksum:TRUE -V -x
tshark.exe -r Lab3_Path_WLAN.pcapng `
  -Y "icmp.type == 8 && !(icmp.type == 11)" `
  -T fields -E occurrence=f -e frame.number -e ip.src -e ip.dst -e ip.ttl
```

附录 D 参考资料

[1] RFC 791 Internet Protocol

https://www.rfc-editor.org/rfc/rfc791.html

[2] RFC 6864 Updated Specification of the IPv4 ID Field

https://www.rfc-editor.org/rfc/rfc6864.html

[3] RFC 792 Internet Control Message Protocol

https://www.rfc-editor.org/rfc/rfc792.html

[4] Microsoft Learn IcmpSendEcho2Ex function

https://learn.microsoft.com/en-us/windows/win32/api/icmpapi/nf-icmpapi-icmpsendecho2ex

[5] Wireshark IPv4 display filter reference

https://www.wireshark.org/docs/dfref/i/ip.html

[6] Wireshark User Guide Checksums

https://www.wireshark.org/docs/wsug_html_chunked/ChAdvChecksums.html